

Lab Assignment

International Cybersecurity and Digital Forensics Academy

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

1. Purpose of the Lab Assignment

The purpose of this lab is to understand how ARP works on a local network, It focuses on identifying normal ARP requests and replies.

The lab also examines how ARP poisoning can change IP to MAC address mappings and packet captures are analyzed to identify signs of suspicious or conflicting ARP activity.

2. Lab Report Structure

The report covers the laboratory setup, network information and evidence integrity, It documents normal ARP behaviour and the analysis of the supplied poisoning capture. The report records suspicious ARP replies and changes in IP-to-MAC mappings.

It also includes ARP-table comparisons, impact assessment and restoration activities.

2.1. Title Page

- Lab Title: LAB 5: ARP Poisoning Forensics
- Student Name: Olanrewaju Wale Mustapha Oladimeji.
- Student ID: 2025/FWSD/11320.
- Course Name: SBT-DF203: Basic Networking Skills for Digital Forensics
- Instructor Name: Aminu Idris
- Date of Submission: 18/09/2026
- Version:

2.2. Executive Summary

This lab investigates ARP poisoning from a digital forensics perspective. A normal ARP process is first observed to establish a baseline, then supplied capture is then examined for conflicting or unsolicited ARP replies.

The investigation looks for cases where one IP address is associated with different MAC

addresses.

The findings help explain how ARP poisoning can support traffic interception and how it can be detected.

2.3. Lab Objectives

The objective is to explain how ARP maps IPv4 addresses to MAC addresses on a LAN. It also involves inspecting ARP tables on Linux and Windows systems.

The lab requires identifying legitimate, unsolicited and conflicting ARP replies. Another objective is to detect when one IP address is associated with multiple MAC addresses.

2.4. Tools and Resources Used

Kali Linux : was used as the main forensic working environment.

Kali Linux was used as the main operating system for performing the practical work.

Apache was used as the local web service on 127.0.0.1:80.

tshark and Wireshark were used to capture and examine the network packets.

Python3 with Scapy was used to generate the small number of authorized SYN packets.

SHA-256 hashing was used to help confirm that the original evidence was preserved.

2.5. Methodology

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
```

```
cd ~/SBT-DF203-Lab5
```

```
pwd
```

```
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
```

```
sudo apt install -y wireshark tshark python3-scapy net-tools
```

```
wget -O evidence/arp.pcap \
```

```
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
```

```
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
```

```
sha256sum evidence/arp.pcap working/arp_working.pcap | tee  
reports/arp_capture_hashes.txt
```

```
ip -br address | tee reports/interfaces.txt
```

```
ip route | tee reports/routes.txt
```

```
ip neigh show | tee reports/arp_table_initial.txt
```

```
(water@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}

(water@kali)-[~]
$ cd ~/SBT-DF203-Lab5

(water@kali)-[~/SBT-DF203-Lab5]
$ pwd
/home/water/SBT-DF203-Lab5

(water@kali)-[~/SBT-DF203-Lab5]
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./reports
./screenshots
./scripts

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ wget -O evidence/arp.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap'
--2026-09-19 14:58:50-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.110.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap      100%[=====>] 1.05K  --.-KB/s  in 0.1s

2026-09-19 14:58:53 (10.1 KB/s) - 'evidence/arp.pcap' saved [1072/1072]

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ ls -lh evidence/
total 4.0K
-rw-rw-r-- 1 water water 1.1K Sep 19 14:58 arp.pcap

(water@kali)-[~/SBT-DF203-Lab5]
$ file evidence/arp.pcap
evidence/arp.pcap: pcapng capture file - version 1.0

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(water@kali)-[~/SBT-DF203-Lab5]
$ ls -lh evidence/ working/
evidence/:
total 4.0K
-rw-rw-r-- 1 water water 1.1K Sep 19 14:58 arp.pcap

working/:
total 4.0K
-rw-rw-r-- 1 water water 1.1K Sep 19 14:58 arp_working.pcap

(water@kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)~[/SBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo        UNKNOWN    127.0.0.1/8 ::1/128
eth0      UP            192.168.244.128/24 fe80::20c:29ff:fe53:93b5/64
docker0   DOWN         172.17.0.1/16

(water@kali)~[/SBT-DF203-Lab5]
$
```

```
(water@kali)~[/SBT-DF203-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.244.2 dev eth0 proto dhcp src 192.168.244.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.244.0/24 dev eth0 proto kernel scope link src 192.168.244.128 metric 100

(water@kali)~[/SBT-DF203-Lab5]
$
```

```
(water@kali)~[/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.244.2 dev eth0 lladdr 00:50:56:ee:25:b8 DELAY
192.168.244.254 dev eth0 lladdr 00:50:56:ee:ec:1d STALE

(water@kali)~[/SBT-DF203-Lab5]
$
```

```
(water@kali)~[/SBT-DF203-Lab5]
$ find ~/SBT-DF203-Lab5 -maxdepth 2 -type f -print
/home/water/SBT-DF203-Lab5/evidence/arp.pcap
/home/water/SBT-DF203-Lab5/working/arp_working.pcap
/home/water/SBT-DF203-Lab5/reports/arp_capture_hashes.txt
/home/water/SBT-DF203-Lab5/reports/arp_table_initial.txt
/home/water/SBT-DF203-Lab5/reports/routes.txt
/home/water/SBT-DF203-Lab5/reports/interfaces.txt

(water@kali)~[/SBT-DF203-Lab5]
$
```

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab5- Olanrewaju Wale Mustapha Oladimeji
Trainee name	Olanrewaju Wale Mustapha Oladimeji
Date and time started	19/09/2026 8:23PM

Field	Student Entry
Evidence file name(s)	evidence/arp.pcap and working/arp_working.pcap
Source or generation method	Supplied arp.pcap capture downloaded from the instructor-provided laboratory source; a working copy was created for analysis.
Original SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
Working-copy SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap
Analysis workstation/V M	Kali Linux virtual machine
Notes on any changes	The original capture was preserved in the evidence folder. A separate working copy was created in the working folder for analysis, and SHA-256 hashes were recorded to verify evidence integrity.

8. Part A - Observe Normal ARP Resolution

1. Identify the victim IP, gateway IP and your interface name.
2. Clear only the specific training neighbor entry where permitted.
3. Start an ARP-only capture, ping the gateway once and stop the capture.
4. Identify the broadcast request and unicast reply.

IFACE=eth0

GATEWAY_IP=\$(ip route | awk '/default/ {print \$3; exit})

sudo ip neigh flush "\$GATEWAY_IP" dev "\$IFACE"

sudo tshark -i "\$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &

sleep 2

ping -c 1 "\$GATEWAY_IP"

wait

ip neigh show | tee reports/arp_table_after_ping.txt

```
(water@kali)-[~/SBT-DF203-Lab5]
$ ip -br address
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        UP          192.168.244.128/24 fe80::20c:29ff:fe53:93b5/64
docker0     DOWN       172.17.0.1/16

(water@kali)-[~/SBT-DF203-Lab5]
$ ip route
default via 192.168.244.2 dev eth0 proto dhcp src 192.168.244.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.244.0/24 dev eth0 proto kernel scope link src 192.168.244.128 metric 100

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ IFACE=eth0

(water@kali)-[~/SBT-DF203-Lab5]
$ GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')

(water@kali)-[~/SBT-DF203-Lab5]
$ echo "Interface: $IFACE"
echo "Gateway: $GATEWAY_IP"
Interface: eth0
Gateway: 192.168.244.2

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ IFACE=eth0

(water@kali)-[~/SBT-DF203-Lab5]
$ GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')

(water@kali)-[~/SBT-DF203-Lab5]
$ echo "Interface: $IFACE"
echo "Gateway: $GATEWAY_IP"
Interface: eth0
Gateway: 192.168.244.2

(water@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show
192.168.244.2 dev eth0 lladdr 00:50:56:ee:25:b8 STALE
192.168.244.254 dev eth0 lladdr 00:50:56:ee:ec:1d STALE

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
[sudo] password for water:

(water@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show
192.168.244.254 dev eth0 lladdr 00:50:56:ee:ec:1d STALE

(water@kali)-[~/SBT-DF203-Lab5]
$ sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &
[1] 20897
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ sleep 2

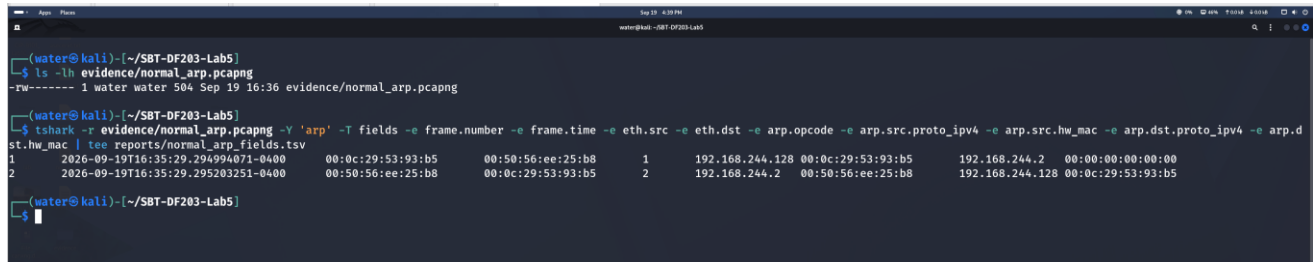
(water@kali)-[~/SBT-DF203-Lab5]
$ ping -c 1 "$GATEWAY_IP"
PING 192.168.244.2 (192.168.244.2) 56(84) bytes of data:
64 bytes from 192.168.244.2: icmp_seq=1 ttl=128 time=0.556 ms

--- 192.168.244.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.556/0.556/0.556/0.000 ms
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_after_ping.txt
192.168.244.2 dev eth0 lladdr 00:50:56:ee:25:b8 STALE
192.168.244.254 dev eth0 lladdr 00:50:56:ee:ec:1d STALE
```

9. Part B - Analyze ARP Request and Reply Fields

```
tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/normal_arp_fields.tsv
```



```
(water@kali) ~/SBT-DF203-Lab5
$ ls -lh evidence/normal_arp.pcapng
-rw-r----- 1 water water 504 Sep 19 16:36 evidence/normal_arp.pcapng
(water@kali) ~/SBT-DF203-Lab5
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.d
st.hw_mac | tee reports/normal_arp_fields.tsv
1      2026-09-19T16:35:29.294994071-0400      00:0c:29:53:93:b5      00:50:56:ee:25:b8      1      192.168.244.128      00:0c:29:53:93:b5      192.168.244.2      00:00:00:00:00:00
2      2026-09-19T16:35:29.295203251-0400      00:50:56:ee:25:b8      00:0c:29:53:93:b5      2      192.168.244.2      00:50:56:ee:25:b8      192.168.244.128      00:0c:29:53:93:b5
(water@kali) ~/SBT-DF203-Lab5
$
```

Field	Normal Request	Normal Reply
Ethernet destination	00:50:56:ee:25:b8	00:0c:29:53:93:b5
Requester MAC	00:0c:29:53:93:b5	00:50:56:ee:25:b8
ARP opcode	1 (Request)	2 (Reply)
Sender protocol address	192.168.244.128	192.168.244.2
Target IP	192.168.244.2	192.168.244.128
Sender hardware address	00:0c:29:53:93:b5	00:50:56:ee:25:b8
Target MAC	00:00:00:00:00:00	00:0c:29:53:93:b5
Target protocol address	192.168.244.2	192.168.244.128
Forensic interpretation	Who has 192.168.244.2?	192.168.244.2 is at 00:50:56:ee:25:b8.

10. Part C - Analyze the Supplied Poisoning Capture

PCAP=working/arp_working.pcap

Inventory all ARP claims

```
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_replies.tsv
```

Summarize IP-to-MAC claims

```
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
```

Look for gratuitous or unsolicited-looking replies

```
tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
-e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
| tee reports/unicast_arp_replies.tsv
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ PCAP=working/arp_working.pcap

(water@kali)-[~/SBT-DF203-Lab5]
$ echo "$PCAP"
working/arp_working.pcap

(water@kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_replies.tsv
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194      00:50:56:86:02:65      136.160.215.15      00:50:56:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15      00:50:56:86:cb:fc      136.160.215.194      00:50:56:86:02:65

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

(water@kali)-[~/SBT-DF203-Lab5]
$
```

```
(water@kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
-e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
| tee reports/unicast_arp_replies.tsv
4      2023-03-11T22:22:44.904123642-0500      136.160.215.194      00:50:56:86:02:65      00:50:56:86:cb:fc
6      2023-03-11T22:22:49.943043799-0500      136.160.215.15      00:50:56:86:cb:fc      00:50:56:86:02:65

(water@kali)-[~/SBT-DF203-Lab5]
$
```

12. Part E - Detection Logic and Timeline

Time	Claimed IP	Claimed MAC	Target	Request Seen First?	Assessment
2023-03-11 22:22:44.904123642	136.160.215.194	00:50:56:86:02:65	136.160.215.15 / 00:50:56:86:cb:fc	Yes – Frame 3	Normal ARP reply; IP-to-MAC mapping

Time	Claimed IP	Claimed MAC	Target	Request Seen First?	Assessment
					is consistent
2023-03-11 22:22:49.943043 799	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194 / 00:50:56:86:02:65	Yes – Frame 5	Normal ARP reply; IP-to-MAC mapping is consistent

- Flag a gateway IP that changes from its legitimate MAC to the analyst/attacker MAC.
- Flag repeated ARP replies without a corresponding recent request.
- Flag one MAC claiming both victim and gateway IP addresses.
- Correlate ARP evidence with packet forwarding, DNS/HTTP anomalies and endpoint ARP-cache changes.

```

(water@kali)~/SBT-DF203-Lab5
$ ls -lh working/arp_working.pcap
-rw-rw-r-- 1 water water 1.1K Sep 19 14:58 working/arp_working.pcap

(water@kali)~/SBT-DF203-Lab5
$ tshark -r working/arp_working.pcap -Y 'arp' -T fields \
-e frame.number -e frame.time \
-e arp.opcode -e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
1 2023-03-11T22:22:03.059513401-0500 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 2023-03-11T22:22:38.192015486-0500 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 2023-03-11T22:22:44.903581555-0500 1 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
4 2023-03-11T22:22:44.904123642-0500 2 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 2023-03-11T22:22:49.943023192-0500 1 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 2023-03-11T22:22:49.943043799-0500 2 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 2023-03-11T22:23:03.059262685-0500 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-11T22:23:06.191965551-0500 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff

(water@kali)~/SBT-DF203-Lab5
$ tshark -r working/arp_working.pcap -Y 'arp.opcode==1' -T fields \
-e frame.number -e frame.time \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
1 2023-03-11T22:22:03.059513401-0500 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 2023-03-11T22:22:38.192015486-0500 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 2023-03-11T22:22:44.903581555-0500 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
5 2023-03-11T22:22:49.943023192-0500 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
7 2023-03-11T22:23:03.059262685-0500 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-11T22:23:06.191965551-0500 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff

(water@kali)~/SBT-DF203-Lab5
$ tshark -r working/arp_working.pcap -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
4 2023-03-11T22:22:44.904123642-0500 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6 2023-03-11T22:22:49.943043799-0500 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65

(water@kali)~/SBT-DF203-Lab5
$ tshark -r working/arp_working.pcap -Y 'arp.opcode==2' -T fields \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort -u
136.160.215.15 00:50:56:86:cb:fc
136.160.215.194 00:50:56:86:02:65

```

13. Restoration and Prevention

Stop any training script first, then clear learned entries on the lab VM

`sudo ip neigh flush all`

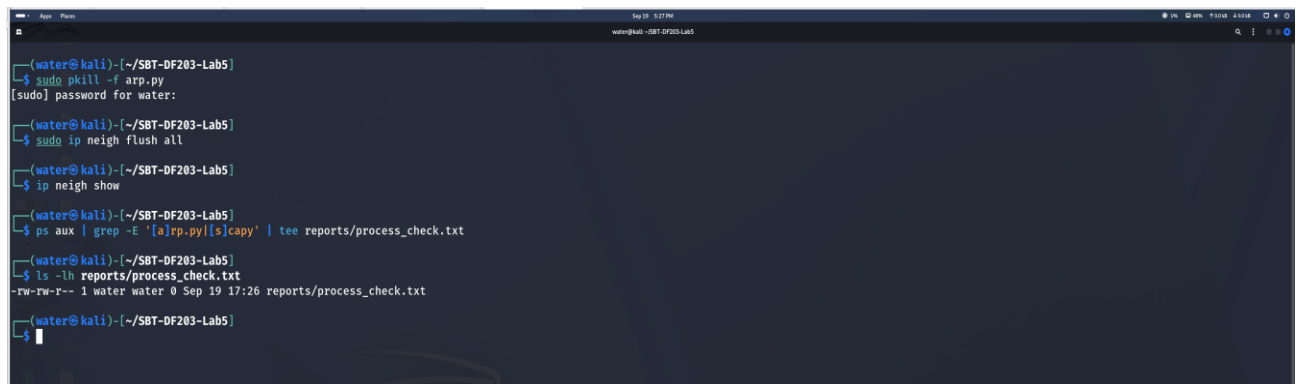
Renew/reconnect the isolated interface as appropriate

`ip neigh show`

Confirm no ARP-poisoning process remains

`ps aux | grep -E '[a]rp.py|[s]copy' | tee reports/process_check.txt`

- Use Dynamic ARP Inspection with DHCP snooping on managed switches where available.
- Segment networks, enable port security and monitor gateway IP-to-MAC consistency.
- Use encrypted application protocols so interception does not expose plaintext.
- Deploy ARP monitoring tools and correlate alerts with switch and endpoint logs.



```
(water@kali)-[~/SBT-DF203-Lab5]
$ sudo pkill -f arp.py
[sudo] password for water:
(water@kali)-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush all
(water@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show
(water@kali)-[~/SBT-DF203-Lab5]
$ ps aux | grep -E '[a]rp.py|[s]copy' | tee reports/process_check.txt
(water@kali)-[~/SBT-DF203-Lab5]
$ ls -lh reports/process_check.txt
-rw-rw-r-- 1 water water 0 Sep 19 17:26 reports/process_check.txt
(water@kali)-[~/SBT-DF203-Lab5]
$
```

2.7. Analysis and Findings

The ARP analysis identified normal request and reply exchanges between the hosts 136.160.215.15 and 136.160.215.194.

Frame 3 requested the MAC address for 136.160.215.194, and Frame 4 provided the correct MAC address 00:50:56:86:02:65.

Frame 5 requested the MAC address for 136.160.215.15, and Frame 6 provided 00:50:56:86:cb:fc.

The IP-to-MAC summary showed one MAC address for each of these two IP addresses in the captured ARP replies.

2.8. Challenges and Solutions

One challenge was that the initial ARP capture file could not be written directly to the evidence folder because of permission errors.

The problem was resolved by saving the capture temporarily in /tmp and then moving it into the correct evidence folder.

2.9. Conclusion

The lab demonstrated how ARP requests and replies can be examined using Tshark.

The captured traffic showed that 136.160.215.15 and 136.160.215.194 had consistent IP-to-MAC mappings in the analyzed replies.

2.10. Recommendations

Assignment like this improved student knowledge and understanding of Network trafficking

2.11. References

Class Video Guide from Founder ICDFa on ARP Poisoning Detection and Wireshark Analysis

Network protocol analyzer guide from <https://www.wireshark.org>